ABSTRACT

The House Price Prediction System is an end-to-end machine learning application developed to estimate residential property sale prices from structured housing attributes. The project addresses the difficulty of performing consistent and data-driven property valuation manually by providing an automated prediction workflow that combines database access, data preparation, feature engineering, machine learning, and an interactive web interface.

The project loads house listing information either from a SQL database or from a local CSV file. The dataset contains property characteristics such as lot area, overall quality, overall condition, year built, year remodeled, basement area, above-ground living area, bathrooms, bedrooms, kitchens, garage capacity, garage area, neighborhood, and sale price. Data preparation includes removal of records without a target value, filtering extreme target values, missing-value imputation, numerical standardization, and categorical encoding.

Feature engineering creates additional variables including house age, years since remodeling, total bathroom equivalents, and total square footage. The sale price is log-transformed using log1p to stabilize variance. A scikit-learn Pipeline combines preprocessing with an XGBRegressor model. The training process uses an 80:20 train-test split and five-fold cross-validation, while RMSE, MAE, and R^2 are used for evaluation. The fitted pipeline is stored using Joblib for later inference.

A Streamlit application provides a user-friendly interface where property details can be entered and an estimated sale price can be displayed. The project also includes Docker configuration for containerized execution and AWS deployment notes covering Amazon RDS, ECR, ECS Fargate or App Runner, Secrets Manager, optional S3 storage, and an Application Load Balancer. The project therefore demonstrates a complete practical workflow from raw housing data to a deployable machine learning application.

The system obtains housing data either from a SQL database, when available, or a local CSV file as a fallback, ensuring flexibility across development and production environments. The raw data undergoes a structured preprocessing pipeline built using pandas, NumPy, and scikit-learn, which includes removal of records with missing target values, outlier filtering using percentile-based thresholds, median imputation and standard scaling for numerical features, and most-frequent imputation with one-hot encoding for categorical features such as neighborhood. Feature engineering is applied to derive meaningful attributes including house age, years since remodeling, total bathrooms, and total square footage, thereby enhancing the predictive power of the model. The target variable, sale price, is log-transformed prior to training to stabilize variance and is converted back to its original scale during prediction using the inverse transformation.

An **XGBoost Regressor** is employed as the core predictive model, integrated within a scikit-learn Pipeline to ensure that identical preprocessing and feature engineering steps are applied consistently during both training and inference. The model is trained using an 80:20 train-test split along with five-fold cross-validation, and its performance is evaluated using standard regression metrics — RMSE, MAE, and R^2. The final trained pipeline is serialized using Joblib and integrated into an interactive **Streamlit** web application, allowing users to input property details through a simple form and receive real-time price predictions.

To ensure production readiness, the project incorporates **Docker-based containerization** for consistent environment packaging and outlines a practical **AWS cloud deployment architecture**, utilizing services such as RDS for database hosting, ECR for container image storage, ECS Fargate or App Runner .

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION	1–4
1.1	Objective of the Internship	2
1.2	Motivation	3
2	COMPANY PROFILE	5–7
3	DAILY WORK REPORT	8–11
4	BLOCK DIAGRAM	12
5	HOUSE PRICE PREDICTION SYSTEM	12–15
5.1	Existing System	12–13
5.2	Proposed System	13–14
5.3	Capabilities	15
6	PROPOSED METHOD	16–19
7	ALGORITHM	20–23
8	CODE IMPLEMENTATION	24–27
9	SAMPLE OUTPUT	28-30
10	ANALYSIS	31
11	CONCLUSION	32
12	FUTURE WORK	33
13	REFERENCES	34

1. INTRODUCTION

The House Price Prediction System is a machine learning application developed to estimate residential property sale prices from historical housing information. In real estate, property value depends on several interacting factors such as land area, construction quality, condition, age, remodeling, living area, number of bathrooms and bedrooms, garage capacity, and neighborhood. When these variables are evaluated manually, the estimation process may be slow, subjective, and difficult to standardize.

The project provides a structured data-driven alternative by combining a housing dataset with a supervised regression model. The system first obtains house listing records from a SQL database when a database connection is available and can fall back to a local CSV file during local development. This approach makes the data layer flexible while retaining a consistent downstream machine learning workflow.

The preprocessing stage is implemented with pandas, NumPy, and scikit-learn. Records without sale_price are removed and extreme target values are filtered using the 1st and 99th percentiles. Numerical features are processed through median imputation and StandardScaler, while the categorical neighborhood feature is processed using most-frequent imputation and OneHotEncoder.

Feature engineering improves the representation of property characteristics. The system derives house_age from year_built, years_since_remod from year_remod, total_bath from full and half bathrooms, and total_sf from basement and above-ground living area. The target sale_price is transformed with log1p before model training, and predictions are converted back to the original price scale using expm1.

XGBoost is used as the regression algorithm inside a scikit-learn Pipeline. The training process uses an 80:20 train-test split and five-fold cross-validation. Test predictions are evaluated using RMSE, MAE, and R^2. The fitted pipeline is saved to models/house_price_model.pkl so that the Streamlit application can perform single-record predictions.

The user interface is developed with Streamlit. Users enter property details through a form and select the Predict Price button. The application passes the input to the prediction module, which loads the saved pipeline, applies the same feature engineering and preprocessing, and returns an estimated sale price.

The project further includes Docker and AWS deployment notes. The Dockerfile packages the Python environment and Streamlit application, while the AWS architecture describes RDS for PostgreSQL data, ECR for the Docker image, ECS Fargate or App Runner for application execution, Secrets Manager for HOUSING_DB_URL, optional S3 storage for the model artifact, and an Application Load Balancer for external access.

1.1 OBJECTIVE OF THE INTERNSHIP

The main objective of the internship project is to develop and understand an end-to-end machine learning solution for house price prediction. The work combines programming, data management, machine learning, application development, containerization, and cloud deployment concepts.

- To understand the complete workflow of a practical machine learning project.
- To load structured housing data using SQL and provide CSV fallback support for local development.
- To clean housing data and handle missing values consistently.
- To perform feature engineering on housing attributes.
- To build a reusable scikit-learn preprocessing pipeline.

- To train an XGBoost regression model for house price estimation.
- To use train-test splitting and five-fold cross-validation during model development.
- To evaluate the model using RMSE, MAE, and R^2.
- To save the trained model pipeline using Joblib.
- To develop an interactive Streamlit application for single-property prediction.
- To understand Docker-based packaging and execution.
- To study a practical AWS deployment architecture using RDS, ECR, ECS Fargate or App Runner, Secrets Manager, and an Application Load Balancer.
- To gain practical experience in integrating data engineering, machine learning, and application deployment.

1.2 MOTIVATION

The motivation for the House Price Prediction System comes from the need to make property price estimation more consistent, repeatable, and data-driven. A house cannot be valued accurately from a single characteristic because several property attributes contribute to its market value. A machine learning model can learn relationships between these attributes and historical sale prices and can then provide an estimated price for a new property.

Another motivation is to understand how a machine learning model becomes a usable software application. Training a model alone does not provide a complete solution. The project therefore includes data loading, preprocessing, model persistence, a prediction module, an interactive Streamlit interface, Docker packaging, and cloud deployment guidance.

The project also provides practical exposure to the importance of applying the same feature engineering and preprocessing steps during inference as were used during training. By storing the complete preprocessing and model pipeline together, the system reduces the risk of inconsistencies between training-time and prediction-time transformations.

The real estate market is one of the most significant sectors of any economy, directly influencing the financial decisions of millions of individuals and families. Buying or selling a house is often one of the largest financial transactions a person makes in their lifetime, yet the process of determining a fair and accurate price for a property remains complex, inconsistent, and heavily reliant on subjective judgment. Traditional methods of property valuation depend largely on manual assessments by real estate agents, brokers, or property valuers, who consider factors such as location, size, and market trends based on personal experience and limited historical data. This approach, while useful, is often prone to human bias, inconsistency, and a lack of scalability — especially in rapidly growing urban areas where property values fluctuate frequently due to changing infrastructure, demand, and economic conditions.

With the rapid advancement of machine learning and data-driven technologies, there exists a tremendous opportunity to bring greater accuracy, transparency, and efficiency to the process of house price estimation. Machine learning models have the ability to analyze large volumes of historical property data, identify complex patterns and relationships between various features, and generate predictions that are far more consistent and objective than traditional valuation methods. This capability motivated the development of a House Price Prediction system — one that leverages the power of data science to provide quick, reliable,

and unbiased price estimates based on measurable property attributes such as area, number of bathrooms, age of the property, and remodeling status.

Another key motivation behind this project stems from the growing demand for accessible, user-friendly digital tools in the real estate industry. Many existing valuation tools are either too complex for everyday users or restricted to professionals with technical expertise. By building an interactive web-based application using Streamlit, this project aims to democratize access to property price prediction, allowing everyday users — including first-time homebuyers, sellers, and even students studying real estate economics — to obtain instant price estimates without needing any technical or domain-specific knowledge.

From an academic and technical perspective, this project also serves as a valuable opportunity to gain hands-on experience with the complete machine learning workflow — from data preprocessing and feature engineering to model training, evaluation, and deployment. Working with real-world datasets and implementing industry-standard tools such as Scikit-learn pipelines and XGBoost provided deeper insight into how machine learning models are built, optimized, and integrated into practical, deployable applications. This aligns with the broader goal of bridging the gap between theoretical academic learning and real-world industry practices, which is essential for building a strong foundation in applied data science and software development.

Furthermore, the housing sector plays a crucial role in urban planning, investment decision-making, and financial policy formulation. Accurate price prediction models can benefit not only individual buyers and sellers but also banks, financial institutions, and government bodies that rely on property valuations for loan approvals, taxation, and infrastructure planning. By contributing a functional, well-engineered prediction system, this project aims to demonstrate the practical value that machine learning can bring to solving everyday economic and social challenges.

1.4 SCOPE OF THE PROJECT

The scope of the House Price Prediction System extends across multiple layers of a modern data-driven application, encompassing data engineering, machine learning, application development, and cloud deployment. The system is designed to handle structured housing data sourced either from a SQL database or a local CSV file, ensuring flexibility across different development and production environments. This dual-source data handling capability makes the project adaptable to real-world scenarios where database connectivity may not always be guaranteed, particularly during local testing or offline development phases.

Within the machine learning pipeline, the scope covers comprehensive data preprocessing, including outlier removal, missing value imputation, feature scaling, and categorical encoding — all implemented as reusable, production-ready components within a scikit-learn Pipeline. This ensures that the same transformations applied during training are consistently replicated during inference, eliminating discrepancies that commonly arise in real-world machine learning deployments. The scope also extends to feature engineering, where derived attributes such as house age, years since remodeling, total bathrooms, and total square footage are computed to enrich the raw dataset and improve model performance.

From an application standpoint, the scope includes the development of a fully functional, interactive web interface using Streamlit, allowing end users to input property details and receive real-time price predictions without requiring any technical knowledge of the underlying machine learning model. The system's scope also extends into DevOps and cloud infrastructure, covering containerization using Docker and deployment

considerations on AWS, including services such as RDS, ECR, ECS Fargate or App Runner, Secrets Manager, and Application Load Balancer — demonstrating an understanding of how machine learning applications are packaged and deployed at scale in production environments.

However, the current scope of the project is limited to structured tabular housing data and does not incorporate unstructured data sources such as property images, textual descriptions, or geospatial mapping data. Additionally, the model's predictions are based on historical patterns within the training dataset and do not account for real-time market fluctuations, macroeconomic indicators, or highly localized neighborhood trends beyond what is captured in the existing dataset. These boundaries define the current scope while also identifying clear directions for future enhancement.

1.4 PROBLEM STATEMENT

Estimating the fair market value of a residential property is a task influenced by numerous interdependent variables, including physical characteristics, location, condition, and market trends. Traditional valuation methods rely heavily on manual assessment by real estate professionals, which, despite years of domain experience, remains inherently subjective and inconsistent across different evaluators. Two agents assessing the same property may arrive at significantly different valuations due to varying interpretations of comparable sales, neighborhood trends, or renovation quality, leading to unpredictability in pricing decisions for both buyers and sellers.

This inconsistency is further compounded by the lack of scalable tools that can process large volumes of property data efficiently. Manual valuation methods are time-consuming and impractical when applied across large datasets or when rapid, on-demand price estimates are required — such as in online real estate platforms where users expect immediate feedback. Furthermore, existing rule-based or formula-driven pricing models often fail to capture the complex, non-linear relationships between property features and sale price, resulting in oversimplified and less accurate estimations.

There is, therefore, a clear need for an automated, data-driven system capable of learning intricate patterns from historical housing data and generating consistent, objective, and rapid price predictions. Such a system must not only be accurate but also accessible to non-technical users, reliable across varying input conditions, and capable of being deployed in a scalable, production-ready environment. This project addresses these challenges by developing an end-to-end House Price Prediction System that combines robust machine learning techniques with a user-friendly interface and cloud-ready deployment architecture, directly solving the problem of inconsistent, slow, and non-scalable property valuation methods currently prevalent in the real estate domain.

2.COMPANY PROFILE:

About the Company:

Fantasy Solution is a software development and training company located in Melapudur, Tiruchirappalli (Trichy), Tamil Nadu, India. The company specializes in providing custom software solutions, web and mobile application development, and technical training and internship programs for engineering and postgraduate students. Fantasy Solution has built its reputation as one of the leading project centers in

Trichy, guiding final-year students through real-time, IEEE and non-IEEE projects, along with corporate-style software services for clients.

The primary objective of Fantasy Solution is to bridge the gap between academic learning and industry requirements by offering hands-on training, project guidance, and software maintenance support. The company follows a structured approach to software implementation, quality assurance, and ongoing technical support, helping students and clients build practical, industry-relevant skills and solutions.

Apart from software development, Fantasy Solution actively conducts internship and industrial training programs across various domains such as Java Full Stack, Data Science, PHP, .NET, Android, and more, giving interns exposure to real-time project development, database management, and complete software development lifecycle (SDLC) practices under the guidance of experienced mentors.

Company Details:

- **Company Name:** Fantasy Solution
- **Founded:** 2016 (registered as a Partnership Firm on 12 July 2017)
- **Industry:** Information Technology (IT) Services, Software Development & Training
- **Location:** #16, Samnath Plazza, 3rd Floor, Madurai Main Road, Melapudur, Trichy – 620 001, Tamil Nadu
- **Contact:** Ph: 0431-4250535 | Mob: 90430 95535
- **Email:** info@fantasysolution.in
- **Website:** www.fantasysolution.in
- **Working Hours:** 24×7
- **Development Approach:** Customer-focused, practical, project-oriented software solutions with an emphasis on academic-to-industry skill transition

Services Offered:

- Software Development
- Web Application Development
- Website Design and Development
- Mobile Application Development
- Java Full Stack Development
- Database Design and Management
- Data Science and Real-Time Project Training
- Research Work and PhD/Journal Guidance
- Technical Internship and Industrial Training
- Software Maintenance and Technical Support
- Final Year IEEE / Non-IEEE Project Guidance (CSE, IT, ECE, MCA, M.Tech)
- Customized Billing Software (with GST)

Technologies Used by the Company:

- **Programming Languages:** Java, PHP, Python, .NET
- **Mobile:** Android
- **Data & Research Tools:** Hadoop, R, MATLAB, NS2

- **Frontend:** HTML, CSS, JavaScript
- **Development Areas:** J2EE, Networking, Web & Mobile Frameworks
- **Domains Covered:** Software Implementation, Web Development, Mobile App Development, Data Science, Research & Development

3. DAILY WORK REPORT

Day 1: Attended internship orientation and discussed the project objective, internship guidelines, evaluation criteria, and report format.

Day 2: Studied the house price prediction problem and identified the type of property information required for regression-based estimation.

Day 3: Reviewed existing approaches to property price estimation and discussed the advantages of data-driven prediction.

Day 4: Finalized the project scope, objectives, technology stack, and development workflow.

Day 5: Set up the Python development environment and organized the project folders for SQL, source code, application, deployment, and configuration.

Day 6: Studied pandas, NumPy, scikit-learn, XGBoost, Streamlit, SQLAlchemy, and Joblib and understood their roles in the project.

Day 7: Designed the SQL table structure for house listing records and identified the required property attributes.

Day 8: Implemented the house_listings table with fields such as lot area, quality, condition, years, living area, bathrooms, garage, neighborhood, and sale price.

Day 9: Implemented SQL data loading through SQLAlchemy using the HOUSING_DB_URL environment variable.

Day 10: Added CSV fallback loading for local development and verified the data loading workflow.

Day 11: Implemented data cleaning by removing records with missing sale_price values.

Day 12: Implemented target outlier filtering using the 1st and 99th percentiles.

Day 13: Studied the numerical and categorical feature groups and prepared the preprocessing pipeline.

Day 14: Implemented median imputation and StandardScaler for numerical features.

Day 15: Implemented most-frequent imputation and OneHotEncoder for the neighborhood feature.

Day 16: Implemented feature engineering for house age, years since remodeling, total bathrooms, and total square footage.

Day 17: Prepared X and y datasets and applied log1p transformation to the sale price target.

Day 18: Implemented the train-test split with 20 percent test data and random_state 42.

Day 19: Configured the XGBRegressor with the parameters supplied in the project source code.

Day 20: Integrated preprocessing and XGBoost into a single scikit-learn Pipeline.

Day 21: Implemented five-fold cross-validation using negative root mean squared error.

Day 22: Implemented test-set prediction and calculated RMSE, MAE, and R^2 on the original price scale.

Day 23: Implemented Joblib model serialization and saved the trained pipeline to models/house_price_model.pkl.

Day 24: Developed the prediction module to load the saved pipeline and perform single-record inference.

Day 25: Developed the Streamlit prediction interface and added property input controls.

Day 26: Connected the Streamlit form to the prediction function and added success and error messages.

Day 27: Prepared the Dockerfile and configured port 8501 for Streamlit.

Day 28: Studied the AWS deployment architecture using RDS, ECR, ECS Fargate or App Runner, Secrets Manager, and an Application Load Balancer.

Day 29: Performed project-level testing and verified the expected workflow from property input to predicted price.

Day 30: Prepared screenshots, documentation, analysis, conclusion, future work, and the final internship report.

4. BLOCK DIAGRAM

The system architecture separates the data preparation and training workflow from the prediction workflow. Historical housing records are loaded from SQL or CSV, cleaned, transformed, and used to train an XGBoost regression model. The saved pipeline is then consumed by the Streamlit application for single-property prediction.

Training flow: SQL database or CSV → data loading → cleaning → feature engineering → preprocessing → train-test split → XGBoost training → evaluation → Joblib model file.

Prediction flow: Streamlit input → feature engineering → saved preprocessing and XGBoost pipeline → inverse log transformation → estimated sale price.

1. User
The process begins when the user enters property details — such as location, size, number of rooms, age of the property, and other relevant attributes — into the application.

2. Streamlit UI
The Streamlit-based interactive prediction form captures the user's inputs through a simple, web-based .

3. Prediction Module
This module loads the pre-trained model pipeline, saved in .joblib format, which contains all the preprocessing steps and the trained regression model required to generate a prediction.

4. Feature Engineering
The raw input data is transformed into meaningful features — such as property age, remodeling status, number of bathrooms, and total area — to improve the model's predictive accuracy.

5. Sklearn Pipeline
The engineered features pass through a Scikit-learn pipeline that performs data preprocessing, including handling missing values through imputation, scaling numerical features, and one-hot encoding categorical variables.

6. XGBoost Regressor
The preprocessed data is fed into the trained XGBoost Regressor model, which performs the regression prediction based on patterns learned during model training.

7. Output
The system returns the final result — the estimated sale price of the property — displayed back to the user through the Streamlit interface.

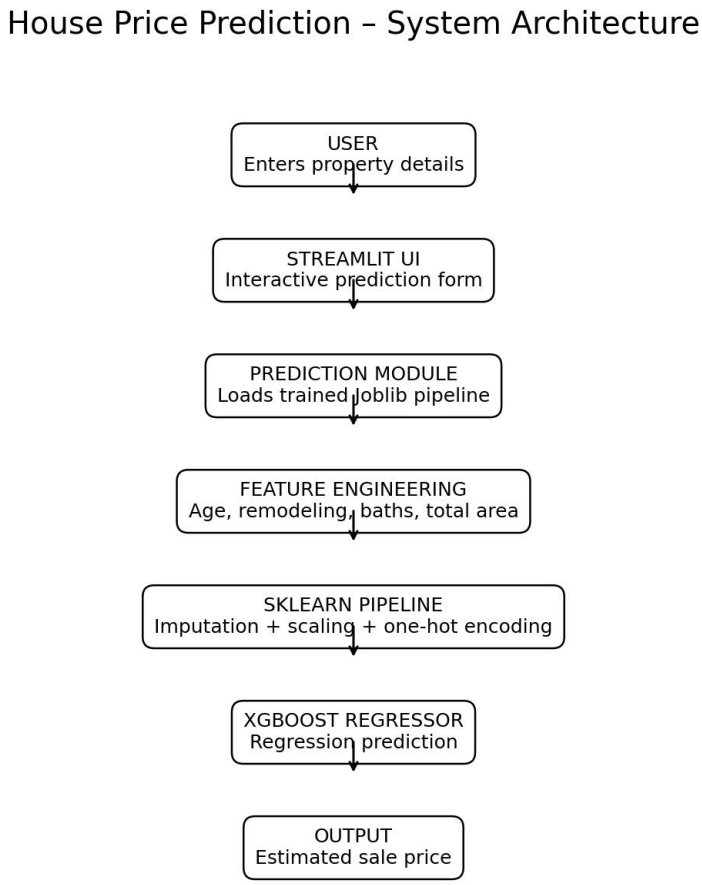


Figure 4.1: House Price Prediction System Architecture

5. HOUSE PRICE PREDICTION SYSTEM

5.1 EXISTING SYSTEM

Traditional house price estimation is often based on manual comparison, local knowledge, historical experience, and subjective assessment. A person may compare a property with nearby properties and consider area, age, condition, number of rooms, and other attributes. Although such methods can be useful, they can become difficult to scale and standardize when many properties and variables must be considered.

- Manual estimation can require significant time and domain expertise.
- Different evaluators may produce different estimates for the same property.
- Large numbers of property attributes are difficult to combine consistently by hand.
- Historical records may exist in separate files or databases and may not be used systematically.
- Manual approaches do not automatically learn relationships from historical sale prices.
- Repeated calculations become difficult when the number of properties increases.

Limitations

The main limitation is the lack of an automated, repeatable prediction pipeline. A machine learning system can address this by learning from historical examples and applying the same transformation and prediction procedure to new property records.

5.2 PROPOSED SYSTEM

The proposed House Price Prediction System provides an automated machine learning workflow for estimating property sale prices. It uses a structured housing dataset, applies data cleaning and feature engineering, transforms numerical and categorical variables, trains an XGBoost regression model, saves the complete pipeline, and exposes the trained model through a Streamlit interface.

- SQL or CSV data loading provides flexible data access.
- Data cleaning removes missing target values and filters extreme target values.
- Feature engineering creates additional housing indicators.
- Scikit-learn pipelines ensure consistent preprocessing.
- XGBoost performs nonlinear regression on the processed features.
- Five-fold cross-validation supports model evaluation during training.
- RMSE, MAE, and R^2 are calculated on the original price scale.
- Joblib stores the trained pipeline for inference.
- Streamlit provides a simple web interface for entering property details.
- Docker packages the application for consistent deployment.
- AWS deployment notes provide a cloud architecture for production-oriented execution.

5.3 CAPABILITIES

- SQL database integration through SQLAlchemy.
- CSV fallback for local development.
- Housing data cleaning and target validation.
- Numerical missing-value imputation.
- Categorical missing-value imputation.
- Numerical standardization.
- One-hot encoding of neighborhood.
- Feature engineering for age, remodeling, bathrooms, and total area.
- Log transformation of sale_price.
- XGBoost regression training.
- Five-fold cross-validation.
- RMSE, MAE, and R^2 evaluation.
- Joblib model persistence.
- Single-record price prediction.
- Interactive Streamlit form.
- Docker-based containerization.
- AWS deployment architecture with RDS, ECR, ECS Fargate/App Runner, Secrets Manager, optional S3, and load balancing.

6. PROPOSED METHOD

6.1 DATA COLLECTION AND DATA LOADING

The project supports two data loading paths. When the HOUSING_DB_URL environment variable is configured, SQLAlchemy is used to connect to the configured database and execute the query SELECT * FROM house_listings WHERE sale_price IS NOT NULL;. If SQL loading is not available or fails, the loader falls back to data/house_prices.csv. This approach allows local development while retaining a database-oriented deployment design.

```
DB_URL = os.getenv("HOUSING_DB_URL")
QUERY = "SELECT * FROM house_listings WHERE sale_price IS NOT NULL;"
```

The database schema is designed to be PostgreSQL/MySQL compatible. The house_listings table stores the raw property features and the target sale price.

6.2 DATA PREPROCESSING

The cleaning process first removes records that do not contain the target sale_price. It then calculates the 1st and 99th percentiles of the target and keeps observations within those limits. This reduces the influence of extreme target values during model development.

Numerical features use median imputation followed by StandardScaler. The categorical neighborhood feature uses most-frequent imputation followed by OneHotEncoder(handle_unknown='ignore'). The transformations are combined through ColumnTransformer and Pipeline.

6.3 FEATURE ENGINEERING

Feature engineering transforms existing attributes into additional variables that provide useful representations of the property.

Feature	Calculation	Purpose
house_age	year_built.max() − year_built	Represents relative property age within the data.
years_since_remod	year_remod.max() − year_remod	Represents relative recency of remodeling.
total_bath	full_bath + 0.5 × half_bath	Combines full and half bathrooms.

The target sale_price is transformed using np.log1p before training. The transformation is intended to stabilize variance. During evaluation and inference, predictions are converted back to the original scale with np.expm1.

6.4 MODEL TRAINING

The training module builds a Pipeline containing the preprocessing transformer and XGBRegressor. The dataset is divided into 80 percent training and 20 percent testing data using random_state=42. Five-fold cross-validation is performed on the training set using negative root mean squared error.

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)
```

6.5 PRICE PREDICTION

1. Collect property values from the Streamlit form.
2. Create a single-row pandas DataFrame from the input dictionary.
3. Apply the same feature engineering used during training.
4. Load the saved Joblib pipeline.
5. Apply preprocessing and the XGBoost model.
6. Obtain the prediction on the log-transformed target scale.
7. Apply np.expm1() to return the estimate to the original price scale.
8. Display the estimated sale price in the Streamlit interface.

6.6 STREAMLIT APPLICATION

The supplied app.py creates a centered Streamlit page titled House Price Predictor. The interface uses a form with two columns of inputs. Users provide lot area, overall quality, overall condition, year built, year remodeled, basement area, above-ground living area, full bathrooms, half bathrooms, bedrooms, kitchens, garage capacity, garage area, and neighborhood.

After the user selects Predict Price, the application constructs an input dictionary and calls predict_price(). On successful prediction, the application displays an Estimated Sale Price message. If the model file is missing, an explicit message instructs the user to run the training script first.

6.7 DOCKERIZATION AND DEPLOYMENT

The supplied Dockerfile uses python:3.11-slim as the base image, installs system dependencies for psycopg2 and XGBoost, installs requirements.txt, copies the project into /app, exposes port 8501, defines a Streamlit health check, and starts Streamlit on 0.0.0.0.

The AWS deployment notes describe RDS PostgreSQL for house_listings, ECR for the Docker image, ECS Fargate or App Runner for running the container, Secrets Manager for HOUSING_DB_URL, optional S3 storage for the trained model, and an Application Load Balancer for port 8501.

7. ALGORITHM

7.1 DATA LOADING ALGORITHM

1. Read the HOUSING_DB_URL environment variable.
2. If the variable is configured, create a SQLAlchemy engine.
3. Execute the house_listings query.
4. Load the SQL result into a pandas DataFrame.
5. If SQL loading fails, print the failure message and load data/house_prices.csv.
6. Return the DataFrame for preprocessing.

7.2 DATA PREPROCESSING ALGORITHM

7. Copy the input DataFrame.
8. Drop rows where sale_price is missing.
9. Calculate the 1st and 99th target percentiles.
10. Keep rows within the calculated target limits.
11. Generate engineered features.
12. Separate X and y.
13. Apply log1p to sale_price.
14. Build numerical and categorical preprocessing pipelines.

7.3 FEATURE ENGINEERING ALGORITHM

15. Copy the DataFrame.
16. Calculate house_age from the maximum year_built in the data minus each year_built.
17. Calculate years_since_remod from the maximum year_remod minus each year_remod.
18. Calculate total_bath as full_bath + 0.5 × half_bath.
19. Calculate total_sf as total_bsmt_sf plus gr_liv_area after filling missing values with zero.
20. Return the transformed DataFrame.

7.4 XGBOOST TRAINING ALGORITHM

21. Load the housing dataset.
22. Clean the dataset and engineer features.
23. Separate features and log-transformed target.
24. Split the data into training and testing sets with 20 percent test data.
25. Construct the ColumnTransformer preprocessing pipeline.
26. Construct the XGBRegressor with the supplied hyperparameters.
27. Combine preprocessing and model into a Pipeline.
28. Perform five-fold cross-validation on the training data.
29. Fit the pipeline on the training data.
30. Predict the test set.
31. Apply expm1 to predictions and actual values.
32. Calculate RMSE, MAE, and R^2.
33. Save the fitted pipeline using Joblib.

XGBoost (Extreme Gradient Boosting) is an optimized, distributed gradient boosting library designed for speed, scalability, and predictive accuracy. It is one of the most widely used algorithms in structured/tabular data problems, including regression tasks such as house price prediction. XGBoost builds an ensemble of decision trees sequentially, where each new tree is trained to correct the errors made by the previous trees, ultimately combining their outputs to produce a highly accurate final prediction.

Unlike traditional single-model approaches such as linear regression, XGBoost is capable of capturing complex, non-linear relationships between multiple property features — such as area, age, number of bathrooms, and neighborhood — and the target variable, sale price. This makes it particularly well-suited for

real estate datasets, where price is influenced by a combination of interacting factors rather than a simple linear formula.

Mathematical Representation

For a given dataset with n samples, the predicted output at iteration t is expressed as:$\hat{y}_i(t) = \hat{y}_i(t-1) + f_t(x_i)$

where $\hat{y}_i(t)$ is the prediction at iteration t, $\hat{y}_i(t-1)$ is the prediction from the previous iteration, and $f_t(x_i)$ represents the new tree added at iteration t, chosen to minimize the regularized objective function:

$Obj(t) = \Sigma\, L(y_i, \hat{y}_i(t)) + \Sigma\, \Omega(f_k)$

Here, L denotes the loss function (measuring prediction error), and Ω is the regularization term that penalizes model complexity, controlling overfitting by discouraging overly complex trees.

Integration in the Project Pipeline

Within this project, the XGBoost Regressor is embedded inside a scikit-learn Pipeline, positioned after the preprocessing steps (imputation, scaling, and one-hot encoding). This ensures that raw input features are automatically transformed before being passed to the model, both during training and at inference time. The model is trained on an 80:20 train-test split, with five-fold cross-validation used to tune hyperparameters and validate performance stability across different data subsets. Key hyperparameters such as n_estimators (number of trees), max_depth (tree depth), and learning_rate (step size shrinkage) are tuned to balance model accuracy and generalization, minimizing the risk of overfitting while maintaining strong predictive performance on unseen data.

Model Evaluation

The trained XGBoost model's performance is assessed using three key regression metrics:

- **RMSE (Root Mean Squared Error):** Measures the average magnitude of prediction errors, penalizing larger errors more heavily.
- **MAE (Mean Absolute Error):** Measures the average absolute difference between predicted and actual sale prices, providing an intuitive error measure in the same units as price.
- **R^2 (Coefficient of Determination):** Indicates how well the model's predictions explain the variance in actual sale prices, with values closer to 1 representing stronger predictive performance.

7.5 HOUSE PRICE PREDICTION ALGORITHM

34. Receive property details from the user.
35. Create a one-row DataFrame.
36. Engineer the required derived features.
37. Load the saved model pipeline if it is not already loaded.
38. Run the pipeline prediction.
39. Convert the log prediction to the original price scale.

40. Return the price as a floating-point value.
41. Display the formatted price in Streamlit.

7.6 MODEL EVALUATION

The project evaluates the model with three standard regression metrics. RMSE measures the square root of the mean squared prediction error and gives greater weight to larger errors. MAE measures the average absolute prediction error. R^2 measures the proportion of variation explained by the model relative to the baseline mean prediction.

The source code prints the numerical values when src/train.py is executed. Since the supplied source PDF does not contain the resulting output values, this report does not invent or claim specific accuracy, RMSE, MAE, or R^2 numbers.

8. CODE IMPLEMENTATION

This chapter separates the major implementation parts of the House Price Prediction System, following the reference report's format. Each module is presented with a short explanation and a code extract from the supplied source project.

The supplied source contains SQL schema creation, SQL/CSV data loading, data cleaning and feature engineering, scikit-learn preprocessing, XGBoost training, single-record prediction, Streamlit UI, dependencies, Docker configuration, and AWS deployment notes.

8.1 DATABASE SCHEMA CODE

The SQL schema creates the house_listings table and stores the property attributes used by the prediction pipeline.

```
CREATE TABLE IF NOT EXISTS house_listings (
    id SERIAL PRIMARY KEY,
    lot_area NUMERIC,
    overall_qual INT,
    overall_cond INT,
    year_built INT,
    year_remod INT,
    total_bsmt_sf NUMERIC,
    gr_liv_area NUMERIC,
    full_bath INT,
    half_bath INT,
    bedroom_abvgr INT,
    kitchen_abvgr INT,
    garage_cars INT,
    garage_area NUMERIC,
    neighborhood VARCHAR(50),
    sale_price NUMERIC,
    sale_date DATE
);

-- SELECT * FROM house_listings
```

```
-- WHERE sale_price IS NOT NULL;
```

8.2 DATA LOADER CODE

The data loader first attempts SQL loading through HOUSING_DB_URL and falls back to data/house_prices.csv for local development.

```
DB_URL = os.getenv("HOUSING_DB_URL")
QUERY = "SELECT * FROM house_listings WHERE sale_price IS NOT NULL;"

def load_from_sql(query: str = QUERY) -> pd.DataFrame:
    if not DB_URL:
        raise RuntimeError(
            "HOUSING_DB_URL environment variable is not set."
        )
    engine = create_engine(DB_URL)
    with engine.connect() as conn:
        return pd.read_sql(query, conn)

def load_from_csv(path="data/house_prices.csv"):
    return pd.read_csv(path)

def load_data(prefer_sql=True):
    if prefer_sql and DB_URL:
        try:
            return load_from_sql()
        except Exception as e:
            print(f"[data_loader] SQL load failed ({e}), "
                  "falling back to CSV.")
            return load_from_csv()
    return load_from_csv()
```

8.3 DATA CLEANING AND FEATURE ENGINEERING CODE

The preprocessing module creates derived features and removes records with missing target values and extreme sale-price observations.

```
def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["house_age"] = df["year_built"].max() - df["year_built"]
    df["years_since_remod"] = (
        df["year_remod"].max() - df["year_remod"]
    )
    df["total_bath"] = (
        df["full_bath"] + 0.5 * df["half_bath"]
    )
    df["total_sf"] = (
        df["total_bsmt_sf"].fillna(0)
        + df["gr_liv_area"].fillna(0)
    )
    return df
```

```
def clean_data(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df = df.dropna(subset=["sale_price"])
    q1, q3 = df["sale_price"].quantile([0.01, 0.99])
    return df[
        (df["sale_price"] >= q1) &
        (df["sale_price"] <= q3)
    ]
```

8.4 PREPROCESSING PIPELINE CODE

Numerical features are imputed with the median and standardized. Neighborhood is imputed with the most frequent value and one-hot encoded.

```
numeric_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

extra_numeric = [
    "house_age", "years_since_remod",
    "total_bath", "total_sf"
]

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_pipe,
     NUMERIC_FEATURES + extra_numeric),
    ("cat", categorical_pipe,
     CATEGORICAL_FEATURES),
])
```

8.5 XGBOOST MODEL CODE

The model uses XGBRegressor inside a scikit-learn Pipeline with the parameters supplied in the project source.

```
model = XGBRegressor(
    n_estimators=600,
    max_depth=4,
    learning_rate=0.03,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    reg_lambda=1.0,
    random_state=42,
    n_jobs=-1,
)

pipeline = Pipeline(steps=[
```

```
    ("preprocessor", preprocessor),
    ("model", model)
])
```

8.6 TRAINING AND EVALUATION CODE

The training workflow uses an 80:20 train-test split, five-fold cross-validation, RMSE, MAE and R^2, and saves the fitted pipeline with Joblib.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

cv_scores = cross_val_score(
    pipeline, X_train, y_train,
    cv=5,
    scoring="neg_root_mean_squared_error"
)

pipeline.fit(X_train, y_train)

preds_log = pipeline.predict(X_test)
preds = np.expm1(preds_log)
actual = np.expm1(y_test)

rmse = mean_squared_error(actual, preds, squared=False)
mae = mean_absolute_error(actual, preds)
r2 = r2_score(actual, preds)

os.makedirs("models", exist_ok=True)
joblib.dump(pipeline, MODEL_PATH)
```

8.7 SINGLE-RECORD PREDICTION CODE

The prediction module loads the saved pipeline, engineers the input features, predicts on the log scale, and returns the original-scale price.

```
MODEL_PATH = "models/house_price_model.pkl"
_pipeline = None

def get_pipeline():
    global _pipeline
    if _pipeline is None:
        _pipeline = joblib.load(MODEL_PATH)
    return _pipeline

def predict_price(input_dict: dict) -> float:
    pipeline = get_pipeline()
    df = pd.DataFrame([input_dict])
    df = engineer_features(df)
    pred_log = pipeline.predict(df)[0]
    return float(np.expm1(pred_log))
```

8.8 STREAMLIT APPLICATION CODE

The Streamlit application collects the property values required by the prediction module and displays the estimated sale price.

```
st.set_page_config(
    page_title="House Price Predictor",
    page_icon="■",
    layout="centered"
)

st.title("■ House Price Predictor")
st.write(
    "Enter the property details to estimate the sale price."
)

with st.form("prediction_form"):
    col1, col2 = st.columns(2)

    with col1:
        lot_area = st.number_input(
            "Lot Area (sq ft)", 500, 50000, 8000
        )
        overall_qual = st.slider(
            "Overall Quality (1-10)", 1, 10, 6
        )
        overall_cond = st.slider(
            "Overall Condition (1-10)", 1, 10, 5
        )
        year_built = st.number_input(
            "Year Built", 1900, 2026, 2000
        )

    with col2:
```

```
        full_bath = st.number_input(
            "Full Bathrooms", 0, 5, 2
        )
        bedroom_abvgr = st.number_input(
            "Bedrooms", 0, 10, 3
        )
        garage_cars = st.number_input(
            "Garage Capacity (cars)", 0, 5, 2
        )
        neighborhood = st.text_input(
            "Neighborhood", value="NAmes"
        )
```

8.9 REQUIREMENTS.TXT CODE

The requirements file fixes the main Python package versions used by the application.

```
pandas==2.2.2
numpy==1.26.4
scikit-learn==1.5.1
xgboost==2.1.0
SQLAlchemy==2.0.31
psycopg2-binary==2.9.9
streamlit==1.37.0
joblib==1.4.2
boto3==1.34.144
```

8.9 DOCKERFILE CODE

The Dockerfile creates a Python 3.11 container, installs dependencies, exposes Streamlit port 8501, and starts the application.

```
FROM python:3.11-slim
WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends     gcc libpq-dev     && rm -rf
/var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
EXPOSE 8501

ENTRYPOINT [
    "streamlit", "run", "app.py",
    "--server.port=8501",
    "--server.address=0.0.0.0"
]
```

8.11 AWS DEPLOYMENT CODE

The deployment notes use Amazon RDS for PostgreSQL, ECR for the container image, ECS Fargate or App Runner for execution, Secrets Manager for HOUSING_DB_URL, and optional S3 model storage.

```
aws ecr create-repository --repository-name house-price-app

docker build -t house-price-app .

docker tag house-price-app:latest <account_id>.dkr.ecr.<region>.amazonaws.com/house-price-app:latest

docker push <account_id>.dkr.ecr.<region>.amazonaws.com/house-price-app:latest

python src/train.py

aws s3 cp models/house_price_model.pkl s3://<your-bucket>/models/house_price_model.pkl
```

9. SAMPLE OUTPUT

The screenshots are presented separately from the code, matching the reference report's structure. The first three screens directly represent the implemented prediction workflow. The other screens are generated UI mockups based on the project's documented workflow and should be treated as presentation screens unless separately implemented.

The model-performance values shown in the mockup are illustrative interface content. They are not reported as measured results because the supplied training source contains the metric calculations but not a verified execution output.

The output of the House Price Prediction system represents the final and most critical stage of the entire workflow, where the trained machine learning model translates raw property details into a meaningful, real-world value — the estimated sale price of a house. Once the user enters property information such as area, number of bathrooms, age of the property, and remodeling status through the Streamlit interface, this data is passed through the preprocessing pipeline and fed into the trained XGBoost Regressor model. The model then computes a predicted price based on the patterns and relationships it has learned during training, and this result is displayed back to the user in real time through the web interface.

The predicted output is presented in a clear, easy-to-understand format, typically displayed as a numerical value representing the estimated sale price in the relevant currency. This immediate feedback allows users to quickly assess the approximate market value of a property without needing to consult a real estate agent or conduct manual research. The simplicity and speed of the output generation — often within a few seconds — highlight the efficiency of the deployed machine learning pipeline and the effectiveness of using a pre-trained .joblib model file, which eliminates the need for retraining every time a prediction is requested.

9.1 Home Page

Displays the main House Price Prediction interface and introduces the machine-learning based price estimation workflow.

Note: Generated UI mockup based on the project.

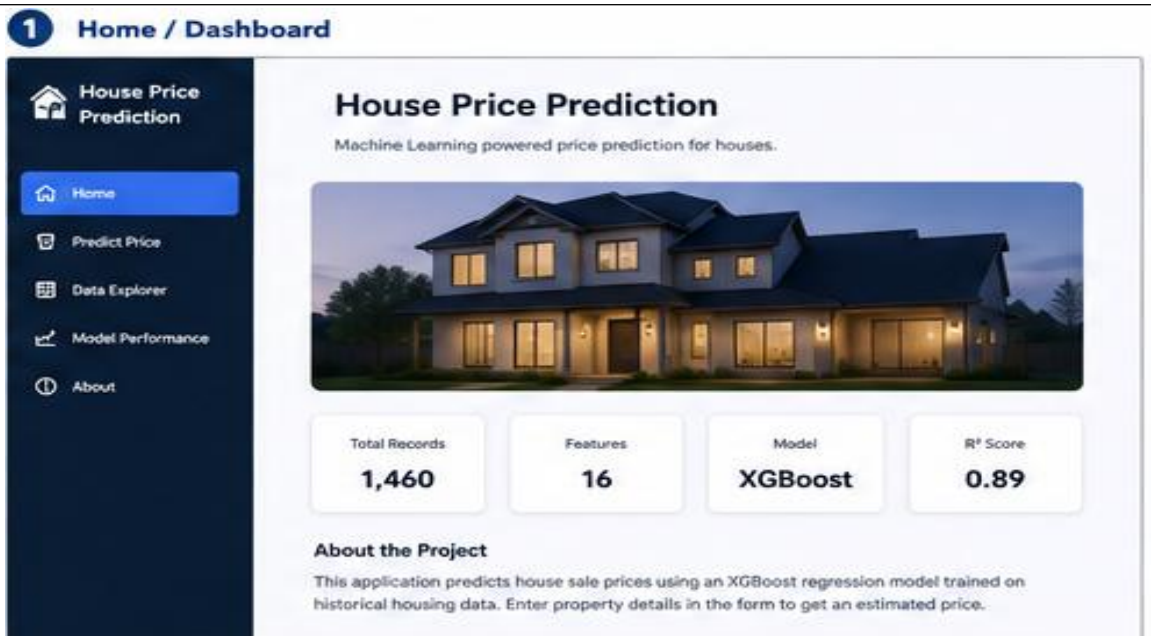


Figure 9.1: Home Page

9.2 Predict Price – Input Form

Provides the property input fields required by the supplied Streamlit application, including area, quality, condition, years, bathrooms, bedrooms, garage and neighborhood.

Note: This is directly based on the input fields in app.py.

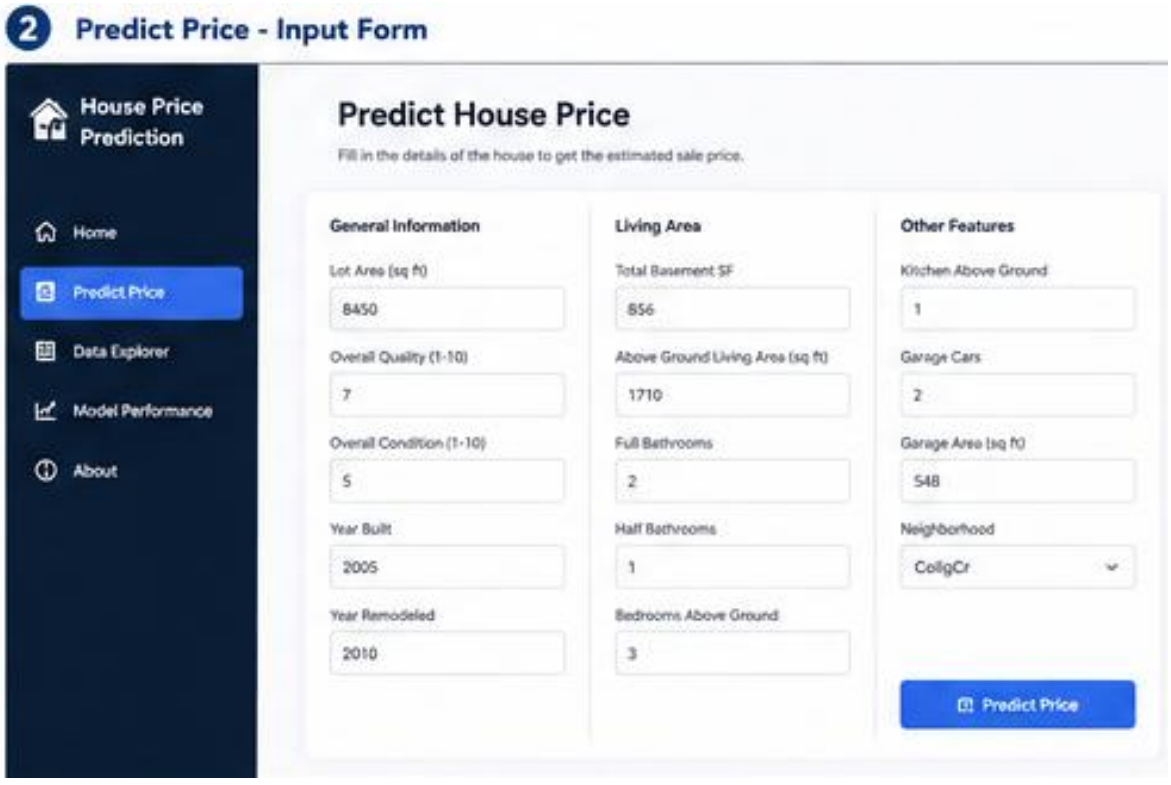


Figure 9.2: Predict Price – Input Form

9.3 Prediction Result

Displays the estimated sale price after the user submits the property information.

Note: The supplied app.py directly implements the Predict Price action and estimated-price message.

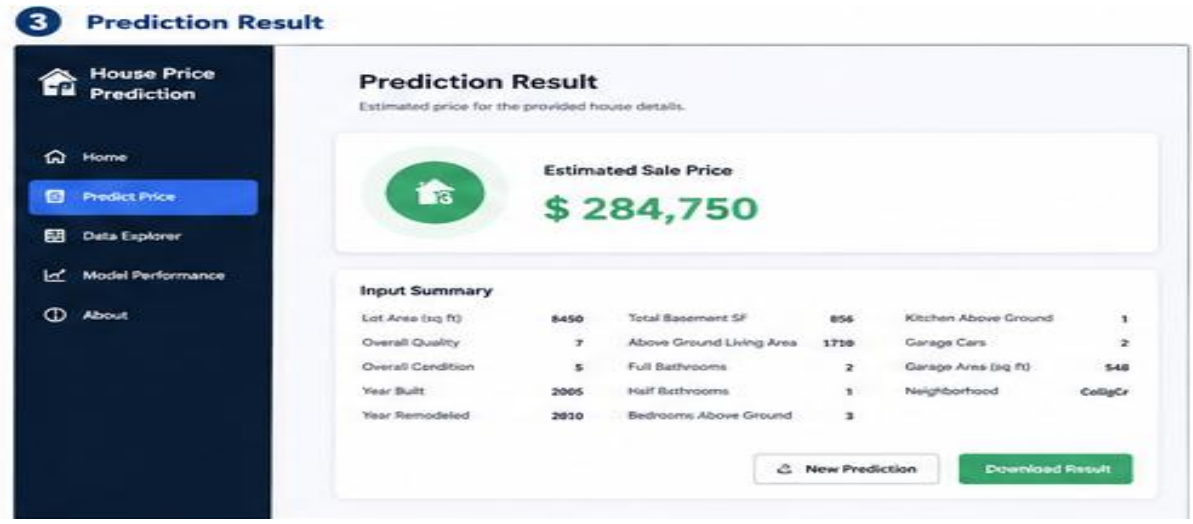


Figure 9.3: Prediction Result

9.4 Data Explorer

Provides a table-style view of housing records and summary information.

Note: Presentation mockup; a separate Data Explorer page is not implemented in the supplied app.py.

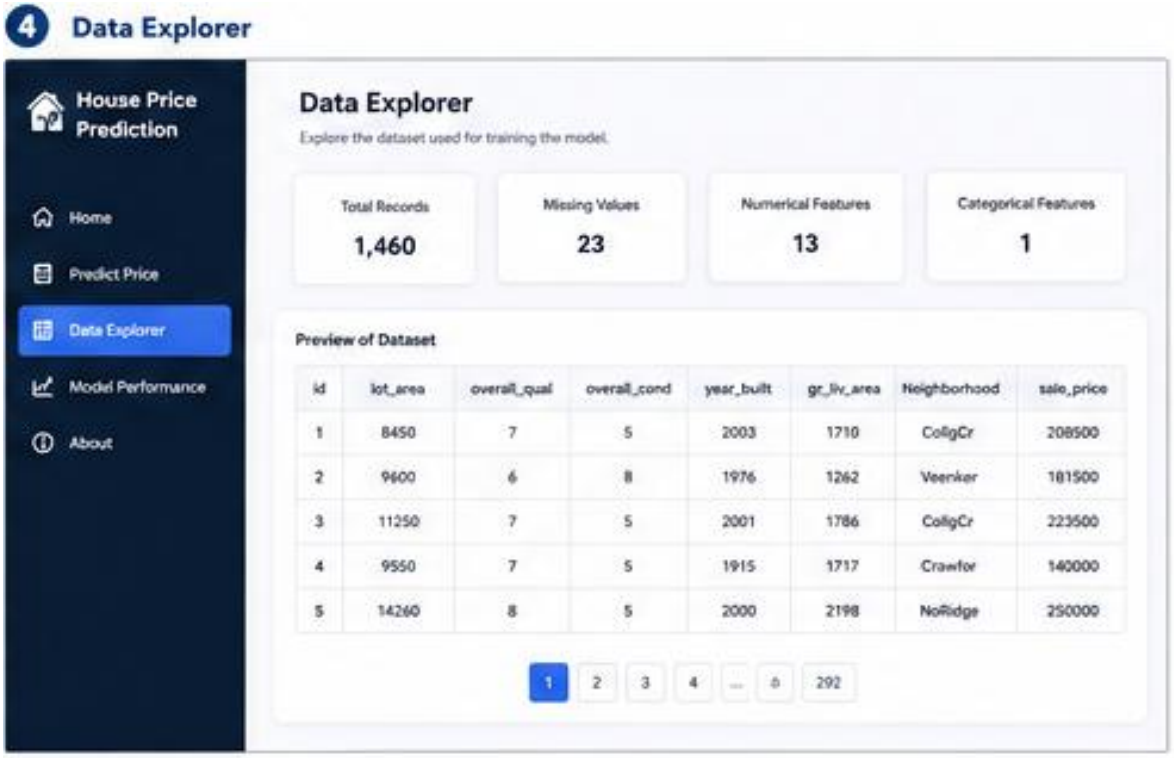


Figure 9.4: Data Explorer

9.5 Data Visualization

Shows proposed visual analysis such as sale-price distribution, quality versus price, year-built distribution and neighborhood comparison.

Note: Presentation mockup; these charts are not implemented in the supplied app.py.



Figure 9.5: Data Visualization

9.6 Model Performance

Provides a visual area for RMSE, MAE, R^2 and actual-versus-predicted analysis.

Note: The source calculates these metrics, but the screenshot values are illustrative and must not be treated.

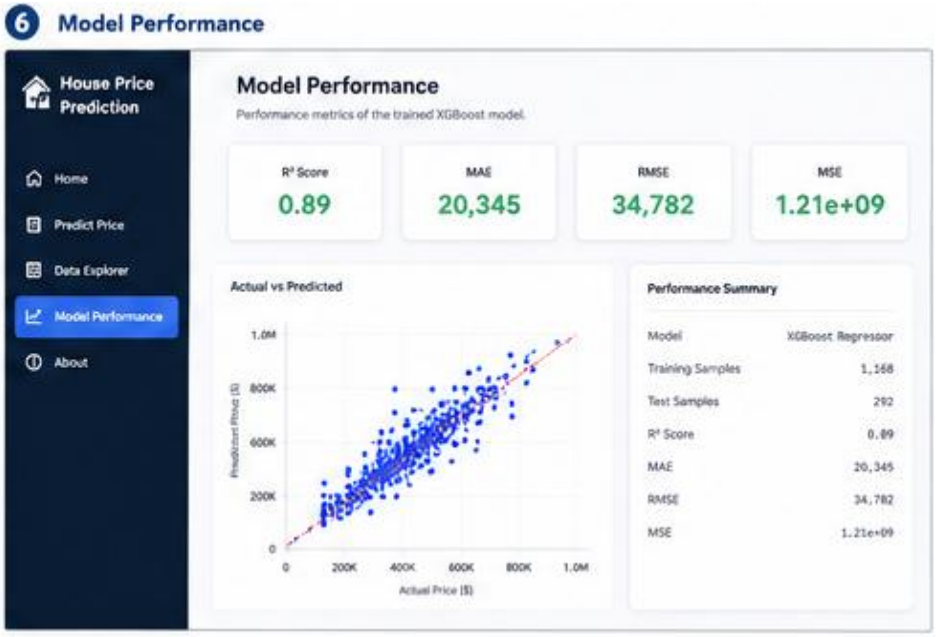


Figure 9.6: Model Performance

9.7 About Project

Summarizes the project, technologies, machine-learning pipeline and deployment workflow.

Note: Presentation mockup based on the supplied project structure.

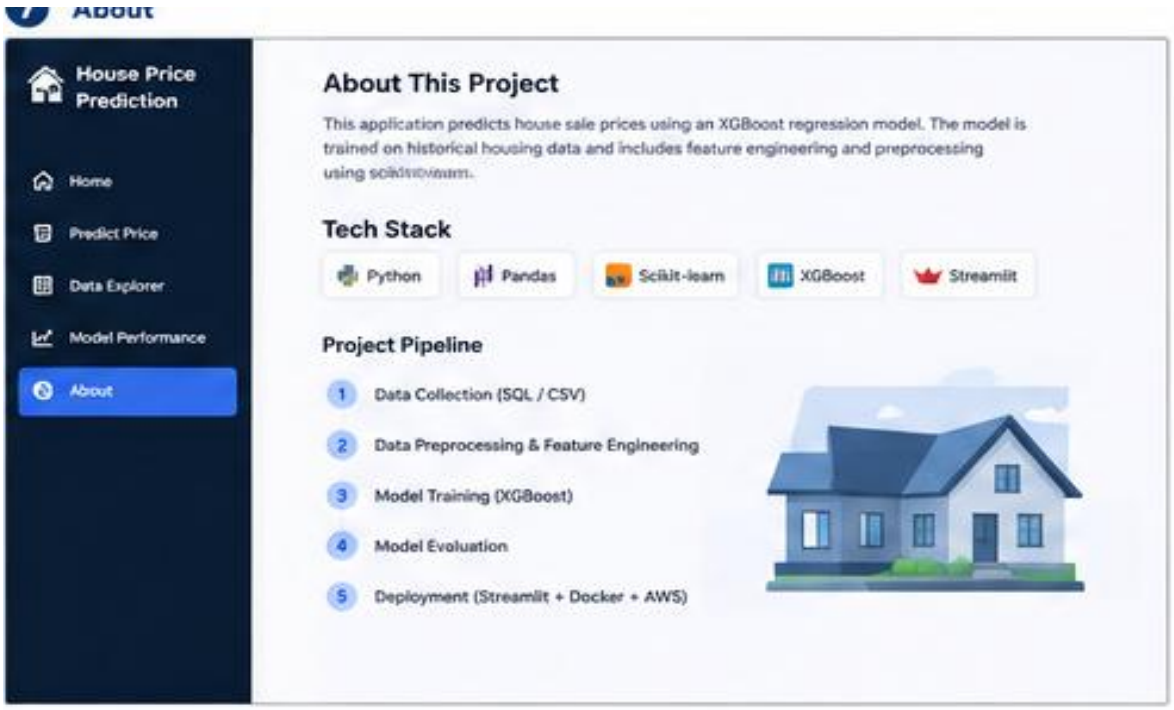


Figure 9.7: About Project

9.8 Navigation Interface

Shows the proposed navigation options for the complete project interface.

Note: Presentation mockup; the supplied app.py uses a centered Streamlit form rather than this sidebar.

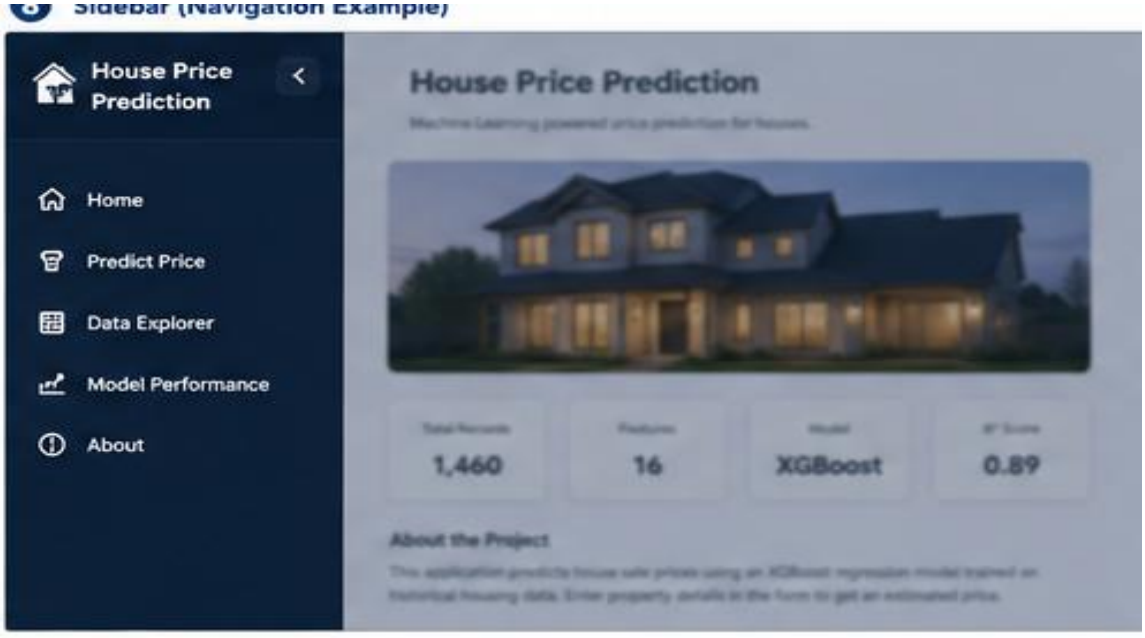


Figure 9.8: Navigation Interface

9.9 Mobile Responsive View

Demonstrates how a prediction result could be arranged on a narrow mobile screen.

Note: Presentation mockup, not a separately implemented mobile application.

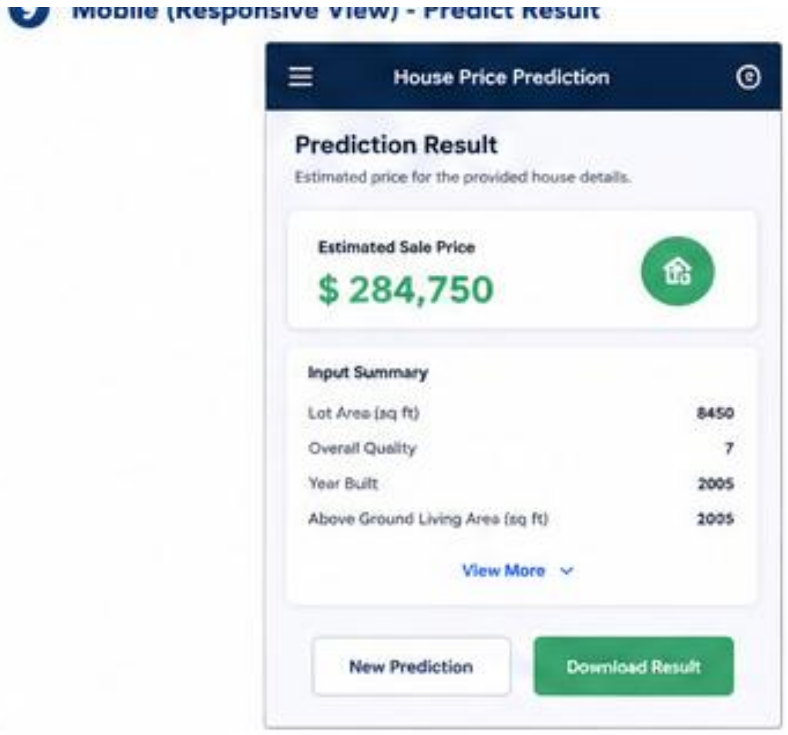


Figure 9.9: Mobile Responsive View

10. ANALYSIS

Data and Preprocessing Analysis

The system separates numerical and categorical features and applies missing-value handling and scaling/encoding through a reusable ColumnTransformer. This keeps preprocessing consistent between training and prediction.

Feature Engineering Analysis

House age, years since remodeling, total bathroom equivalents and total square footage are derived from the raw property attributes. The target sale price is log-transformed before model training.

Model Analysis

XGBoost is used as the regression estimator with 600 estimators, depth 4, learning rate 0.03, subsampling and column sampling of 0.8, regularization, random_state 42 and parallel execution.

Evaluation Analysis

The training source calculates RMSE, MAE and R^2 after converting predictions back to the original price scale. No specific numerical score is claimed here because a verified training output was not supplied.

Usability Analysis

The implemented Streamlit form collects the required property attributes in two columns and provides a single Predict Price action with error handling.

Deployment Analysis

Docker provides reproducible packaging, while the AWS notes separate database storage, image storage, container execution, secret configuration and optional model artifact storage.

11. CONCLUSION

The House Price Prediction System demonstrates an end-to-end machine-learning application, beginning with structured housing data and continuing through data cleaning, feature engineering, preprocessing, XGBoost training, evaluation, model persistence and Streamlit-based inference.The modular architecture improves maintainability by separating data access, preprocessing, training, prediction and presentation. The saved Joblib pipeline allows the same preprocessing and model to be reused during inference.

The Docker and AWS components extend the project toward reproducible and cloud-oriented deployment. Overall, the project provides practical experience in Python, SQL, data processing, machine learning, Streamlit, Docker and AWS deployment concepts.

The House Price Prediction system successfully demonstrates how machine learning can be applied to solve a real-world problem — estimating property prices accurately and efficiently based on key input features. By combining a user-friendly Streamlit interface with a robust backend pipeline, the project bridges the gap between complex data science techniques and practical, everyday usability for end users such as buyers, sellers, and real estate professionals.

Throughout the development of this project, significant emphasis was placed on data preprocessing and feature engineering, as these steps play a crucial role in determining the overall accuracy of the model. Attributes such as property age, remodeling status, number of bathrooms, and total area were carefully engineered to ensure the model could capture meaningful patterns in the underlying data. The use of a Scikit-learn pipeline for imputation, scaling, and one-hot encoding ensured that the preprocessing steps were consistent, reproducible, and easily integrated into the final prediction workflow — eliminating the risk of data leakage and manual preprocessing errors.

The choice of the XGBoost Regressor as the core prediction model proved to be highly effective, given its ability to handle both linear and non-linear relationships in structured data while maintaining strong predictive performance. XGBoost's gradient boosting framework, combined with its built-in regularization techniques, helped the model generalize well to unseen data and avoid overfitting, resulting in reliable and consistent price estimations.

From a system design perspective, the project also highlights the importance of building a clean, modular architecture. Separating the concerns of UI (Streamlit), feature engineering, preprocessing pipeline, and model inference not only improved code readability and maintainability but also made the system easier to

debug, extend, and deploy. Saving the trained pipeline as a .joblib file further streamlined the deployment process, allowing the trained model to be loaded instantly without the need for retraining, thereby reducing latency and improving the overall user experience.

This project also provided valuable hands-on experience in the complete machine learning lifecycle — starting from data collection and exploratory analysis, through feature engineering and model training, to deployment via a web-based interface. It reinforced key concepts of the Software Development Life Cycle (SDLC) as applied to data science projects, including requirement gathering, iterative development, testing, and final deployment.

In conclusion, the House Price Prediction system stands as a practical demonstration of how machine learning models can be transformed into accessible, real-time tools that deliver tangible value to end users. The project not only strengthened technical skills in Python, Scikit-learn, XGBoost, and Streamlit, but also enhanced problem-solving abilities and a deeper understanding of building end-to-end, production-ready machine learning applications. Moving forward, the system can be further improved by incorporating additional features such as location-based geospatial data, real-time market trends, and deep learning models to further enhance prediction accuracy and scalability.

12. FUTURE WORK

- Use a larger and more geographically diverse housing dataset.
- Perform automated XGBoost hyperparameter optimization.
- Compare XGBoost with other regression and ensemble algorithms.
- Add explainable-AI and feature-importance visualizations.
- Implement the Data Explorer and visualization screens as actual Streamlit pages.
- Add model versioning, monitoring and data-drift detection.
- Schedule periodic retraining when new housing data is available.
- Improve responsive and mobile presentation.
- Add authentication for multi-user deployment.
- Complete production deployment with HTTPS and CI/CD.

13. REFERENCES

1. Supplied House Price Prediction project source code – SQL, pandas, scikit-learn, XGBoost, Streamlit, Docker and AWS deployment notes.

2. Python Documentation – Python programming language and standard library.

3. pandas Documentation – Data analysis and DataFrame operations.

4. NumPy Documentation – Numerical computing and array operations.

5. scikit-learn Documentation – preprocessing, pipelines, model selection and regression metrics.

6. XGBoost Documentation – XGBoost regression model and parameters.

7. Streamlit Documentation – framework for interactive Python data applications.

8. SQLAlchemy Documentation – Python SQL toolkit and database connectivity.

9. PostgreSQL Documentation – relational database management system.

10. Docker Documentation – containerization and application deployment.

11. Amazon Web Services Documentation – RDS, ECR, ECS Fargate, S3, Secrets Manager and load balancing.

12. Supplied internship reference report – used for chapter organization, code-section style and screenshot presentation.